

P-set a

Name: Zuha Aqib

ERP ID: 26106

Semester: 4

Date: 5th May 2024

Task: Problem Set 2

Course: Theory of Automata

Teacher: Sir Shahid Hussain

Q1. Prove that the class of context-free languages is not closed under either intersection or complementation. [Hint: the class of context-free languages is known to be closed under union]

To prove something is not in the class of a language, we will disprove it using an example.

INTERSECTION:

- so we will get two languages L_1 and L_2 and make a language L_3 where

$$L_3 = L_1 \cap L_2$$

L_3 is the intersection of L_1 and L_2 , we will then try and make a CFL / prove

L_3 is a CFL.

- Example:

$$L_1 = \{ 0^n 1^n 0^m \mid m, n \geq 0 \}$$

So L_1 is a context free language as we can define a context free grammar as we can define a context free grammar for it:

$$S \rightarrow \epsilon \mid S_1 S_2$$

$$S_1 \rightarrow 0 S_1 1 \mid \epsilon$$

$$S_2 \rightarrow 0 \mid \epsilon$$

so in L_1 , it forms strings $0^n 1^n 0^m$ where the first 0 and 1 have equal copies but the second 0 may or may not be equal.

$$L_2 = \{ 0^n 1^m 0^m \mid m, n \geq 0 \}$$

So L_2 is also a context free language as we can define a context free grammar for it:

$$S \rightarrow S_1 S_2$$

$$S_1 \rightarrow 0 \mid \epsilon$$

$$S_2 \rightarrow 1 S_2 0 \mid \epsilon$$

so in L_2 , it forms strings $0^n 1^m 0^m$ where the 1 and second 0 have equal copies but the first 0 may or may not be equal to them.

So we form language L_3 , where L_3 is the intersection of L_1 and L_2 ,

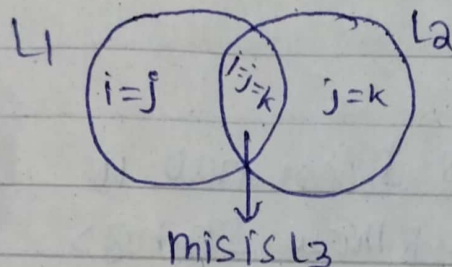
$$L_1 \\ 0^n 1^n 0^m \\ 0^i 1^j 0^k$$

$i=j$, k may be equal

$$L_2 \\ 0^n 1^m 0^m \\ 0^i 1^j 0^k$$

$j=k$, i may be equal

So L_3 is $0^i 1^j 0^k$ where $i=j=k$



$$L_3 = \{ 0^n 1^n 0^n \mid n \geq 0 \}$$

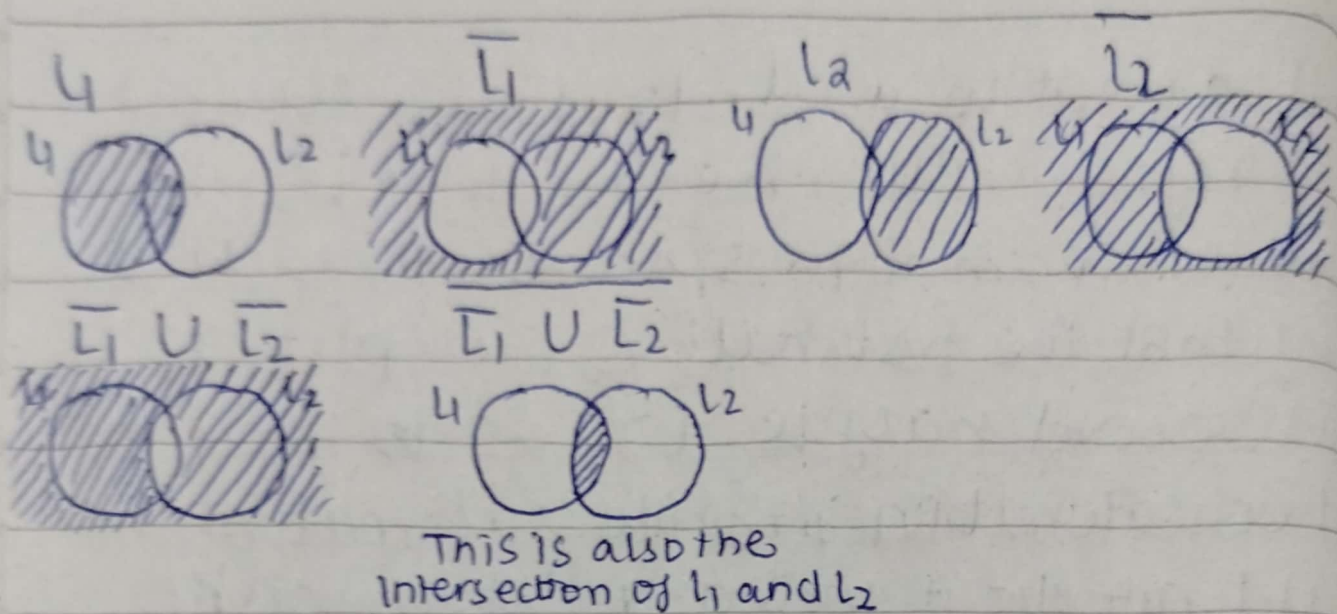
But, L_3 is not a context free ^{language} grammar as we cannot define a context free grammar or a pushdown automaton for it. ^{prove using pumping lemma.}
 ~ Hence proved, the class of context free languages is not closed under intersection.

COMPLEMENT:

Proof by Contradiction: The ^{intersection} ~~complement~~ of L_1 and L_2 can be written as

$$L_1 \cap L_2 = \overline{L_1} \cup \overline{L_2}$$

as



So

$$L_1 \cap L_2 = \overline{\bar{L}_1 \cup \bar{L}_2}$$

~~so if the R.H.S, the intersection, would~~
 so if the L.H.S, the complement, would
 be closed under content free languages
 men [union is also closed] the intersection
 should also be closed. But, we proved
 above that intersection is not closed.

Therefore our assumption that complement
 is closed is wrong and it is contradicted.
 ~ Hence complement is also not
 closed.

Q2. Show that, every infinite Turing-recognizable language has an infinite Turing decidable subset.

Ans. Let L be an infinite Turing Recognizable language. If L is Turing Recognizable, there must be an enumerator for it. Let this enumerator be E .

A enumerator E' exists such that it runs E and prints only those strings printed by E that are longer than, previous string printed by E' .

E' prints those strings that belong to L . E' is an enumerator for L .

A Turing machine D that runs E' . If any string printed by E' matches the input string then accept. If any string printed by E' is longer than input string, reject. Since, all shorter strings have been already printed, this implies that input

string doesn't belong to $D(L)$.

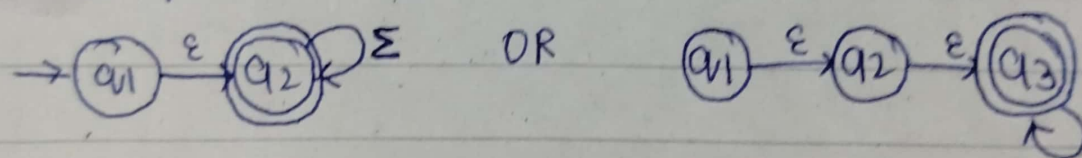
D is a decider and it only accepts strings that belong to L .

Therefore, $D(L)$ is a decidable subset for L . Hence, every Turing recognizable language has a Turing decidable subset.

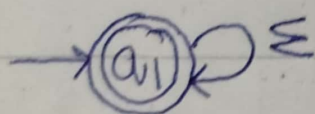
Q3. Let $ALL_{DFA} = \{ \langle A \rangle : A \text{ is a DFA and } L(A) = \Sigma^* \}$. Prove that ALL_{DFA} is decidable.

ALL_{DFA} consists of all those DFA's whose language is Σ^* . Meaning those DFA's accept all types of strings, i.e. all inputs are accepted. Even an empty string ϵ is accepted as $\epsilon \in \Sigma^*$.

If it was ALL_{NFA} , we could have made an example NFA as



But this is a DFA, [no epsilon transition]



There will be only one state which will be the start and ^[final] accepting state, as it is a DFA to accept Σ^* [sigma star]. It accepts every permutation and combination of its input string.

- So when I start my Turing machine

ALL_{DFA} and get a DFA A , I will run the following algorithm steps:

Steps:

1. Decode A from its encoding and change to a DFA. If not a valid DFA, reject.
2. Mark the start state of A
3. Continue marking all the states in such a way that the state that has any transition coming into it from any other already marked state, it will be marked.
4. keep repeating step 3 until no new states are marked.
5. Accept when all accepting states are marked. Reject Otherwise.

Hence shown it is decidable.

Proof #2:

~~The previous proof~~ Construct a DFA B such that the language of B is the complement of A . Use the TM T which is E_{DFA} [Theorem] on input $\langle B \rangle$. If T accepts, it means B is empty and A is Σ^* . Accept. Else when T rejects, reject.

Q4. let $T = \{a^i b^j c^k : i, j, k \in \mathbb{N}\}$. Prove that T is countable, here a, b, c are symbols from some alphabet.

Ans. Countable means there must be some one-to-one correspondence to the count. A set is countable if a set is either finite or has the same cardinality as the set of positive integers.

An easy way to prove it is countable is to sum i, j, k . As i, j, k are in $\mathbb{N}$, the smallest value each of them can have is

1. So the smallest sum will be

$$i=1, j=1, k=1 = \{1, 1, 1\} \quad s = 3$$

so s starts from 3. We find all the triples which make s ,

$$s=3 \quad \{1, 1, 1\} \quad 1$$

$$s=4 \quad \{1, 2, 1\}, \{1, 1, 2\}, \{2, 1, 1\} \quad 3$$

$$s=5 \quad \{1, 2, 2\}, \{2, 1, 2\}, \{2, 2, 1\},$$

$$\{1, 1, 3\}, \{1, 3, 1\}, \{3, 1, 1\}$$

$$\vdots$$

and so on.

So for each s , there will be a finite number of triples of $\{i, j, k\}$ who make the sum s . Each number $s \in \mathbb{N}$.

So we enumerate the triples with sum zero (none), sum equal to 1 (none), to 2 (none), equal to 3 (1), equal to 4 (3), equal to 5 (6), and so on. Thus we enumerate all these triples in T . Hence our set will be finite for every $s \in \mathbb{N}$.

Hence our set is finite and according to the definition, countable.

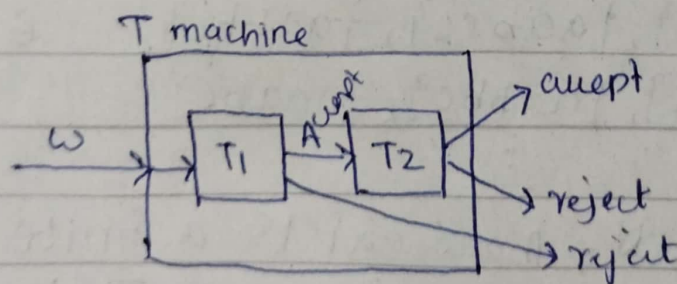
We can think of this the same way as length of string l ,

- | | | |
|-------|--|---|
| $l=3$ | $\{abc\}$ | 1 |
| $l=4$ | $\{abbc\}, \{abcc\}, \{aabc\}$ | 3 |
| $l=5$ | $\{abbcc\}, \{aabcc\}, \{aabbcc\},$
$\{abccc\}, \{abbbc\}, \{aaabc\}$ | 6 |

$\vdots$
hence for $l \in \mathbb{N}$, there exists a finite number of strings in the form of T , that have length l . Hence it is finite and countable.

Q5. If L_1 and L_2 are two Turing Recognizable languages, show that their concatenation is also Turing recognizable. To do this, assume that you have Turing machines recognizing L_1 and L_2 . Then explain how to construct a Turing machine that recognizes $L_1 L_2$. Describe your construction in English. Feel free to give your Turing machine extra working tapes (if needed).

Ans. So we have a machine T_1 who recognizes L_1 and a machine T_2 who recognizes L_2 . We will create a machine T that recognizes $L_1 L_2$.



So speaking in layman terms, we have a string w which is passed to T_1 , it scans half of the string [the L_1 part], if it rejects, reject w completely, else pass the L_2 part [remaining].

and accept w as $L_1 L_2$ if L_2 accepts else reject. But, the problem is, we don't know where to split w in such a way that the first half of the split is L_1 and second half is L_2 . So ~~to~~ when we did concatenation in finite automata, we could not do it using a deterministic finite automaton so we did it using a non-deterministic finite automaton in which we created branches from the accepting states of M_1 to the start state of M_2 . [M_1 recognizes $L(L_1)$ and M_2 recognizes $L(L_2)$].

For this, we can either use a NTM [non deterministic Turing machine] or a deterministic TM with multiple tapes. We will go with the tapes option.

So if we have a string w , ~~no more~~ ~~at~~ with length n , there are $(n+1)$ ways to split it in half. For example, a string with length 5 can be split in the 6 ways:

P-Set (continue)

length 5 in $(5+1=6)$ ways:

/	□	□	□	□	□	□	01234
□	/	□	□	□	□	□	0.1234
□	□	/	□	□	□	□	01.234
□	□	□	/	□	□	□	012.34
□	□	□	□	/	□	□	0123.4
□	□	□	□	□	/	□	01234.

So the / (slash) means a split. This means "0.1234" that the 0th character is the first half of the split and the 1234th characters are the second half of the split. So we put each of these strings on each of the tapes — hence we require $(n+1)$ tapes. We represent the split using a # symbol [a delimiter] where this symbol is not in the alphabet. An example can be,

$w = \text{"cat"} \quad |w| = 3 \quad \text{tapes} = 4$

tape 1	□ # c a t
tape 2	c # a t
tape 3	c a # t
tape 4	c a t # □

Where □ is a blank symbol not in the alphabet.

So the machine runs this way:

~~to get the first tape~~ Total tapes = $n+2$

① tape 1 consists of the input w written on it

② create $(n+1)$ splits of w and write each split on each tape. Put a delimiter $\#$ between each split.

③ Get the second tape which has the first split. Put the head at the start of the tape. If the head has an ^{blank}~~empty~~ symbol, check if T_1 's start state is an accepting state. If yes, move ahead. If not, ^{go to step 6} ~~reject~~. If it is not an ^{blank}~~empty~~ symbol, traverse the head's contents in T_1 . Keep traversing until a delimiter arrives or the head comes to an accepting state. If the head comes to an accept state, move ahead. ~~If~~ If the delimiter is read & it is not ^{an} an accept state, ~~reject~~ go to step 6.

④ The remaining part of the string [after the delimiter] will be traversed on T_2 . If the string is a blank symbol, check if T_2 's start state is an accepting state. If yes, accept w . If not, ^{go to step 6} ~~reject~~. If not a blank symbol, traverse the input in

T2 until a blank symbol or an accept state comes. If an accept state comes, accept, if a blank symbol comes and it is not an accept state, go to step 6.

- ⑤ If it is "accepted" that means w is a concatenation of L_1 and L_2 . accept and stop running.
- ⑥ Move to the next tape and re-run again from step 3. If all tapes are run, and none have been "accepted", then reject w .

Hence we have designed a Turing Recognizable machine for the Concatenation of two Turing Recognizable languages.